

	Universidad Autónoma de Manizales Facultad de Ingeniería Programa de Ingeniería de Software 1	Fecha: 8 de octubre de 2025
---	--	---

Sabor UAM (Aplicación académica del ciclo de vida del software)

Ingeniería de Software 1
Diana Marcela Henao Montoya

Maribel Ceballos Castrillon

Elizabeth Meneses Muñoz

Maria Paz Puerta Acevedo

Heily Yohana Rios Ayala

Manizales, Colombia
10 Noviembre de 2025

Sabor UAM (Aplicación académica del ciclo de vida del software)

Presentado por:

Maribel Ceballos Castrillón – Código: 10202400268

María Paz Puerta Acevedo – Código: 10202400258

Elizabeth Meneses Muñoz – Código: 10202400276

Heily Yohana Ríos Ayala – Código: 10202400265

Docente: Diana Marcela Henao Montoya

Curso: Ingeniería de Software 1

Semestre: 2025-03

Índice

1. Justificación	4
2. Objetivos	5
Objetivo General	5
Objetivos específicos	5
3. Stakeholders	6
4. Recolección de Requisitos	7
4.2 Listado de Requerimientos	10
Técnica (como – quiero – para) historias de usuario	12
5. Casos de Uso	13
5.1. Identificación de casos de uso	13
5.2. Especificación de casos de uso	14
5.3. Diagrama de casos de uso	33
6. Planificación del Proyecto	34
6.1. Técnica de planificación elegida	34
6.2. Planificación de entregables	34
6.3. Cronograma	37
7. Evidencias del Desarrollo	39
7.1. Mockups e imágenes de las funcionalidades	39
7.2. Maquetación HTML de las funcionalidades	52
7.3. Captura de código frontend relevante	58
8. Diseño de Base de Datos	70
8.2. Descripción de entidades, atributos y tipos de datos	71
8.3. Relaciones entre entidades	72
8.4. Script SQL con datos de prueba	73
9. Resultados y Presentación del Proyecto	73

	4
9.1. Funcionalidades implementadas	73
9.2. Flujos de funcionamiento de la plataforma	73
9.3. Presentación en PowerPoint (evidencia de exposición en aula)	73
9.4. Infografía resumen del proyecto	73
10. Conclusiones	73
11. Referencias Bibliográficas	75

1. Justificación

El principal motivo por el cual queremos desarrollar este proyecto es la necesidad de encontrar, gestionar e identificar los lugares de alimentación cercanos disponibles para la comunidad de la Universidad Autónoma de Manizales (UAM). Actualmente, nos damos cuenta que los estudiantes, docentes y demás personal de la institución no siempre están informados sobre los lugares de comida más cercanos, sus horarios de atención, menús, entre otros servicios adicionales. Lo que hace que en ocasiones no puedan comer por cuestiones de tiempo. Además, se ha identificado la importancia de contar con un usuario que funcione como administrador que permita mantener actualizada la información de los lugares de comida, asegurando que los usuarios siempre tengan acceso a datos confiables y verificados.

Para solucionar esta problemática proponemos la elaboración de una página web que contenga información sobre los lugares de alimentación de la universidad, esta contendrá como se dijo antes la información gastronómica de la comunidad UAM y tendrá un sistema para gestionar los diferentes productos que se ofrecerán, pero solo el administrador podrá gestionarlos, actualizarlos y verificar que los datos sean correctos, además de poder comprobar la disponibilidad de los productos y su estado, garantizando así que la información que se muestre a los usuarios se encuentre actualizada.

El grupo objetivo de este proyecto está conformado principalmente por los estudiantes de la UAM, quienes son los que más requieren este tipo de información debido a la organización de sus actividades académicas. No obstante, también se verán beneficiados los docentes, el personal administrativo y demás miembros de la institución, quienes igualmente necesitan acceder de manera rápida y sencilla a opciones de alimentación confiables y accesibles.

2. Objetivos

Objetivo General

Informar a toda la comunidad UAM sobre las diferentes opciones gastronómicas en varios lugares de todo el campus a través de la página web SaborUAM, una página diseñada para que los usuarios puedan descubrir y explorar los sitios donde se ofrecen comidas y otros productos. Los usuarios podrán ver la descripción, los horarios de atención, la disponibilidad del producto, la ubicación y el precio de cada producto, con el fin de ofrecer una buena experiencia.

Objetivos específicos

- Crear una función la cual permita autenticar y autorizar a los usuarios que ingresen e inicien sesión con sus datos personales para garantizar que la información sea accesible exclusivamente para miembros de la comunidad UAM.
- Implementar una función la cual tenga la posibilidad de permitir al administrador y solo a él gestionar las diferentes tareas tales como: eliminar, agregar y actualizar la información de los diferentes lugares con cada uno de sus diferentes productos garantizando la información publicada.
- Garantizar que todos los usuarios puedan tener la información sobre los lugares y su estado como por ejemplo: cerrado o abierto en tiempo real, también incluyendo la posibilidad de visualizar las diferentes promociones que ofrece ya sea semanal o mensual según la estrategia de cada lugar.

3. Stakeholders

- Estudiantes

Descripción: Miembros de la universidad que usan la página para buscar lugares de comida cercanos.

Justificación: Son los principales usuarios y quienes más frecuentan los sitios de comida.

- Docentes

Descripción: Profesores que pueden consultar opciones de alimentación durante su jornada laboral.

Justificación: Usan el sistema como apoyo para encontrar opciones de comida dentro o cerca de la universidad.

- Administrativos

Descripción: Personal de oficina y servicios de la universidad.

Justificación: También requieren opciones de comida y hacen parte de la comunidad UAM.

- Sitios de comida cercanos

Descripción: Restaurantes, cafeterías, panaderías u otros negocios que ofrecen alimentos alrededor de la universidad.

Justificación: Son los proveedores de los productos que se muestran en el sistema, sin ellos no habría oferta que consultar.

4. Recolección de Requisitos

4.1 Para la recolección de requisitos del sistema se aplicaron dos técnicas principales: encuestas en línea y entrevistas directas.

- Estudiantes

Técnica aplicada: Encuesta en línea.

Evidencia: Se realizó un formulario con preguntas sobre la frecuencia con la que consumen en sitios cercanos, cuál es la información más importante para ellos (precio, menú, ubicación, horarios), y saber un poco más sobre su opinión personal de nuestra futura página web.



- **Docentes**

Técnica aplicada: Encuesta en línea.

Evidencia: Se realizó un formulario con preguntas sobre la frecuencia con la que consumen en sitios cercanos, cuál es la información más importante para ellos (precio, menú, ubicación, horarios), y saber un poco más sobre su opinión personal de nuestra futura página web.



- **Administrativos/Personal**

Técnica aplicada: Encuesta en línea.

Evidencia: Se realizó un formulario con preguntas sobre la frecuencia con la que consumen en sitios cercanos teniendo en cuenta sus horarios laborales, cuál es la información más importante para ellos (precio, menú, ubicación, horarios), y saber un poco más sobre su opinión personal de nuestra futura página web.



- **Sitios de comida cercanos (vendedores)**

Técnica aplicada: Entrevista directa.

Evidencia: Se realizaron entrevistas con dueños o encargados de los locales para obtener información sobre menús, precios, promociones, horarios y disposición, la cual era información que aparecerá en la aplicación.

[Preguntas para dueños de los negocios \(entrevista\)](#)

4.2 Listado de Requerimientos

Redacción formal y estructurada

- Requerimientos funcionales

1. El sistema debe permitir al administrador eliminar productos del menú.
2. El sistema debe permitir al administrador actualizar la información de un producto existente.
3. El sistema debe permitir al administrador agregar información de un producto al menú seleccionando una categoría.
4. El sistema debe permitir cerrar sesión.
5. El sistema debe permitir iniciar sesión con correo electrónico y contraseña.
6. El sistema debe permitir al usuario registrarse proporcionando datos como nombre, correo y contraseña.
7. El sistema debe permitir al usuario consultar el menú.
8. El sistema debe permitir al usuario consultar detalles de los productos como una descripción breve, el precio, horario de atención y ubicación.
9. El sistema debe permitir al usuario seleccionar un producto del menú.
10. El sistema debe permitir al usuario seleccionar una categoría en el menú (desayunos, almuerzos, u otros productos)

- Requerimientos no funcionales
 1. La interfaz de la página web para que el usuario pueda obtener información sobre los productos y filtrar las búsquedas debe ser intuitiva y fácil de usar.
 2. El sistema debe cargar en un tiempo máximo de 3 segundos al iniciar.
 3. El sistema debe permitir que los usuarios disfruten de una disponibilidad de la página web 24 horas los 7 días de la semana.
 4. El sistema debe asegurar que los datos y la información que se muestre de cada lugar con sus datos sea la correcta.
 5. El sistema debe tener los colores que representen a la comunidad UAM, como el color amarillo, azul, blanco y negro.
 6. El sistema debe soportar hasta 30 usuarios conectados simultáneamente sin afectar el rendimiento ni el tiempo de carga de la página.
 7. El sistema debe proteger los datos de contacto del usuario frente a personas externas a la universidad.
 8. El sistema debe recuperarse automáticamente en menos de 10 segundos si se presenta un fallo como la pérdida de conexión sin afectar la información del usuario.
 9. El sistema debe tener un logo y letra representativos de la página web.
 10. El sistema debe ser totalmente compatible con el navegador Google Chrome, asegurando que todas sus funciones y elementos visuales se ejecuten correctamente.

Técnica (como – quiero – para) historias de usuario

1. Como administrador, quiero iniciar sesión en el sistema para gestionar los productos de la página web.
2. Como administrador, quiero cerrar sesión para asegurar que nadie más pueda acceder a mis funciones.
3. Como administrador quiero eliminar productos del menú, para evitar que los usuarios consulten opciones que ya no se encuentran disponibles en la página web.
4. Como administrador, quiero agregar información de un producto al menú seleccionando una categoría, para organizar los productos de manera adecuada y facilitar su visualización para los usuarios.
5. Como administrador quiero actualizar la información de los productos, para que los usuarios puedan estar informados de nuevos precios, horarios y ubicación.
6. Como usuario nuevo, quiero registrarme en el sistema proporcionando mi nombre, correo institucional y contraseña, para crear una cuenta y acceder a las funciones disponibles en la página web.
7. Como usuario, quiero iniciar sesión en el sistema para buscar los productos disponibles.
8. Como usuario, quiero consultar el menú para ver las opciones disponibles (desayuno, almuerzo u otros productos).
9. Como usuario, quiero consultar la ubicación y precio de los productos para saber en qué establecimientos adquirirlos.

10. Como usuario, quiero consultar los horarios de atención para planear cuándo puedo ir a comprar los productos.

5. Casos de Uso

5.1. Identificación de casos de uso

Actores:

- **Usuario UAM:**
Persona de la comunidad UAM que usa la página SaborUAM para ver los lugares donde se venden comidas o productos en el campus.
- **Administrador:**
Usuario que puede agregar, eliminar o actualizar los productos que aparecen en la página.

Casos de uso identificados:

Actor: Usuario UAM

1. Registrarse
- 2 . Iniciar sesión
3. Cerrar sesión
4. Consultar menú.
5. Seleccionar producto.
6. Consultar detalles del producto (como descripción, precio, horario de atención, disponibilidad y ubicación).

Actor: Administrador

1. Agregar producto.
2. Eliminar producto
3. Actualizar producto

5.2. Especificación de casos de uso

ID	AGR-001
Fecha	01/10/2025
Título	Agregar producto
Versión	01
Autores	Heily Yohana Rios Ayala
Actores	Administrador
Descripción	Permite que el administrador registre un nuevo producto en el sistema especificando sus datos

	como nombre, precio, categoría entre otros.
Precondición	El administrador tuvo que haber ya iniciado sesión en el sistema.
Flujo Normal	1. El administrador selecciona la opción "Agregar producto". 2. El sistema muestra el formulario de registro. 3. El administrador debe ingresar los datos del producto. 4. El sistema valida que todos los campos estén llenos. 5. El sistema guarda el nuevo producto en la base de datos. 6. El sistema confirma la creación del producto.
Flujo alterno	• Si el administrador no completa los campos obligatorios, el sistema muestra un mensaje de error. • Si ya existe un producto con el mismo nombre, el sistema notifica que ya existe

	uno igual.
Poscondiciones	El nuevo producto queda registrado y disponible para consulta y gestión en el sistema actualizarlo, eliminarlo entre otras.

ID	ELI-002
Fecha	01/10/2025
Título	Eliminar producto
Versión	01
Autores	Maria Paz Puerta Acevedo
Actores	Administrador
Descripción	Permite que el administrador elimine un producto

	del sistema por su identificador único.
Precondición	• El producto a eliminar debe existir previamente en el sistema.
Flujo Normal	1. El administrador selecciona la opción “Gestionar productos” . 2. El administrador selecciona la opción “Eliminar producto” . 3. El sistema muestra un campo en el cual se ingresa el identificador único del producto. 4. El administrador selecciona el producto que desea eliminar. 5. El sistema muestra un mensaje de confirmación para eliminar el producto. 6. El administrador selecciona la opción de confirmación. 7. El sistema elimina el producto de su base de datos. 8. El sistema muestra un mensaje de que el producto fue eliminado.

Flujo alternativo	• Si el producto no existe, el sistema debe mostrar un mensaje indicando que el producto no se encuentra y no se realiza la eliminación. • Si el administrador selecciona la opción de cancelar, el sistema deberá interrumpir el proceso de eliminación y volver a la pestaña de eliminación.
Poscondiciones	El producto que fue seleccionado por el administrador se elimina correctamente en el sistema y deja de estar visible en el listado de productos disponibles.

ID	ACT-003
Fecha	01/10/2025
Título	Actualizar Producto

Versión	01
Autores	Maria Paz Puerta Acevedo
Actores	Administrador
Descripción	Permite que el administrador pueda modificar la información de un producto o servicio que ya exista en el sistema como tal dando a conocer sus datos tales como: precio, descripción, si maneja promociones cuales serian para que la información publicada se garantice.
Precondición	El producto a modificar debe existir previamente en el sistema.
Flujo Normal	1. El administrador selecciona la opción "Gestionar productos" . 2. El administrador selecciona el lugar que contiene el producto a actualizar. 3. El sistema muestra los productos que tiene disponible el lugar. 4. El Administrador selecciona el producto a

	modificar. 5. El administrador modifica los datos deseados del producto (promociones y sus estados, precio, descripción etc) 6. El administrador guarda los cambios. 7. El sistema valida que todos los campos e información sean válidos. 8. El sistema actualiza la información del producto y muestra un mensaje de confirmación.
Flujo alterno	• -El sistema identifica campos vacíos o información no válida esto hace que el sistema muestre un mensaje de error y permite al administrador corregir nuevamente los datos. • - El administrador no guarde los nuevos datos y el sistema no actualice la información •
Poscondiciones	La información del producto (nombre, precio, descripción, etc.) queda modificada y persistida en el

	sistema, siendo visible de inmediato para todos los usuarios UAM en la página de consulta.
--	--

ID	CON-004
Fecha	01/10/2025
Título	Consultar detalles del producto
Versión	01
Autores	Maria Paz puerta Acevedo
Actores	Usuario UAM
Descripción	Permite ver la información completa de un producto como su descripción, precio, horario de atención y ubicación.

Precondición	• El usuario debe haber iniciado sesión en el sistema. • Debe existir al menos un producto registrado con precio en el sistema.
Flujo Normal	1. El usuario selecciona un producto del menú. 2. El sistema muestra la información completa: • Descripción • Precio • Horario de atención • Ubicación del sitio 3. El usuario puede volver al menú o consultar otro producto.
Flujo alternativo	• Si el producto no tiene información registrada, el sistema muestra: "Información no disponible."

	• Si ocurre un error al cargar los datos, el sistema muestra: “No fue posible mostrar la información del producto.”
Poscondiciones	• El usuario visualiza la información del producto seleccionado. • El sistema queda listo para que el usuario consulte otro producto o regrese al menú.

ID	SEL-005
Fecha	01/10/2025
Título	Seleccionar producto

Versión	01
Autores	Heily Yohana Rios Ayala
Actores	Usuario UAM
Descripción	Permite al usuario elegir un producto del menú para ver su información completa.
Precondición	• El usuario debe haber consultado el menú. • Debe existir al menos un producto en la categoría seleccionada.
Flujo Normal	1. El usuario ve la lista de productos de la categoría seleccionada. 2. Selecciona un producto. 3. El sistema prepara la información para mostrar los detalles.

Flujo alterno	• Si no hay productos disponibles en la categoría, el sistema muestra: “No hay productos en esta categoría.” • Si ocurre un error al seleccionar, el sistema muestra: “No se pudo seleccionar el producto, inténtelo nuevamente.”
Poscondiciones	El sistema queda listo para mostrar la información del producto elegido.

ID	INI-006
Fecha	02/10/2025
Título	Iniciar sesión

Versión	01
Autores	Elizabeth Meneses Muñoz
Actores	usuario UAM
Descripción	Permite al actor autenticarse en el sistema mediante sus credenciales (usuario,contraseña) para acceder a las funcionalidades y servicios de la página web.
Precondición	El actor debe estar registrado en el sistema y tener credenciales válidas.
Flujo Normal	1. El actor accede a la página de login 2. El actor ingresa su usuario y contraseña 3. El actor selecciona el botón “iniciar sesión” 4. El sistema verifica las credenciales en la base de datos si las credenciales son válidas el sistema el sistema valida esto e identifica el rol de actor 5. El sistema crea una sesión activa y redirige al actor a la vista principal ya después de

	identificar su rol y sus funcionalidades.
Flujo alterno	Credenciales inválidas si las credenciales no coinciden, el sistema muestra el mensaje “ usuario o contraseña incorrectos “ y permite al actor reintentar.
Poscondiciones	El actor queda en un estado de autenticado con un rol de usuario o administrador .

ID	CER-007
Fecha	02/10/2025
Título	Cerrar sesión
Versión	01

Autores	Elizabeth Menes Muñoz
Actores	usuario UAM
Descripción	Permite al usuario UAM finalizar su sesión autenticada en la plataforma Sabor UAM. Este proceso revoca las credenciales activas en la página web.
Precondición	El administrador y el usuario UAM ya debe de haber iniciado sesión correctamente en el sistema.
Flujo Normal	El actor selecciona la opción de “cerrar sesión”. El sistema verifica si la sesión está activa. El sistema destruye o elimina la sesión.
Flujo alterno	sesión expirada por ejemplo si el sistema detecta que la sesión ha caducado anteriormente por inactividad o por otros temas, notifica al actor y el paso 3.
Poscondiciones	El Administrador pierde el acceso al panel de gestión y a todas las funciones de edición.

ID	CON-008
Fecha	02/10/2025
Título	Consultar Menú
Versión	01
Autores	Heily Yohana Rios Ayala
Actores	Usuario UAM
Descripción	Permite ver las categorías de productos disponibles en la página (Desayunos, Almuerzos y Otros productos).
Precondición	El usuario debe haber iniciado sesión.

	Debe existir al menos una categoría con productos disponibles.
Flujo Normal	El usuario inicia sesión. Selecciona la opción “Consultar menú”. El sistema muestra las categorías disponibles (Desayunos, Almuerzos, Otros). El usuario elige una categoría. El sistema muestra los productos correspondientes a esa categoría.
Flujo alterno	Si no hay categorías disponibles, el sistema muestra: “No hay productos disponibles en este momento.” Si ocurre un error al cargar la información, el sistema muestra: “Error al mostrar el menú, inténtelo nuevamente.”

ID	REG-009
Fecha	02/10/2025
Título	Registrarse
Versión	01

Autores	Elizabeth Meneses Muñoz
Actores	Usuario UAM
Descripción	Permite a un nuevo miembro de la comunidad UAM crear una cuenta en la plataforma para acceder a las funciones de interacción (como dejar comentarios y valoraciones), pasando de un estado anónimo a un estado de Usuario UAM autenticado.

Precondición	El actor debe ser un usuario nuevo y no debe tener una cuenta registrada previamente con el mismo correo electrónico o usuario.
Flujo Normal	1. El actor selecciona la opción de "registrarse" desde la página de inicio. 2. El sistema presenta el formulario de registro ejemplo: nombre, correo, contraseña, documento, tipo de documento. 3. El actor completa los campos requeridos. 4. El sistema valida los datos 5. El sistema notifica al usuario el registro exitoso
Flujo alterno	Correo ya registrado si el correo electrónico ya existe , el sistema muestra el mensaje "ya existe una cuenta con este correo"

Poscondiciones	El actor queda en un estado de usuario registrado y , opcionalmente , autenticado.
----------------	--

5.3. Diagrama de casos de uso



6. Planificación del Proyecto

6.1. Técnica de planificación elegida

Para el desarrollo del proyecto Sabor UAM se usa la metodología ágil Scrum, con ayuda de la herramienta Jira que nos ayuda a la gestión y seguimiento de tareas.

Scrum permite organizar el trabajo por sprints semanales donde se definen metas y tareas específicas. En cada Sprint Planning, el equipo establece las actividades principales, asigna responsabilidades y prioriza las tareas según su relevancia. Con la herramienta Jira se pueden ver las tareas en un tablero, seguir el avance y registrar los entregables de cada sprint. Esta técnica ayuda a trabajar de forma ordenada y así poder cumplir con el cronograma del proyecto.

6.2. Planificación de entregables

Entregables

- 1. Documentación del proyecto:** Este entregable incluye la descripción completa de lo que se tratará el proyecto, redacción del documento de visión, identificación de los requerimientos funcionales y no funcionales, el diagrama de casos de uso, los objetivos del proyecto, entre otros. Estas tareas nos ayudarán a identificar la arquitectura del software y las restricciones que tendrá nuestro sistema SaborUAM.

2. **Mockups:** En esta parte se creará un diseño previo de como se verá el sistema, es decir se usarán elementos gráficos como imágenes, diseño de botones, distribución de la información, entre otros. Estos se harán por medio de herramientas como canva, draw.io, balsamiq y otras herramientas que nos faciliten su realización.
3. **Diseño de la interfaz de usuario:** En este entregable se empezará crear el sistema a partir de los mockups aprobados por el cliente y los integrantes de nuestro proyecto.
4. **Código fuente:** Este entregable incluye la implementación completa del sistema SaborUAM utilizando HTML, CSS y JavaScript, con todos los módulos, funciones y elementos necesarios para cumplir con los requerimientos definidos. El código estará debidamente documentado y comentado para facilitar su comprensión, mantenimiento y futuras actualizaciones.
5. **Pruebas:** En este entregable se hace la verificación del funcionamiento del sistema SaborUAM según lo definido en el documento de visión y demás alcances acordados. Se realizan pruebas funcionales y de aceptación, registrando posibles errores o ajustes necesarios para asegurar que el sistema funcione.
6. **Bitácoras de cada sprint:** Contendrán el registro detallado de las actividades realizadas durante cada sprint, los avances logrados, los problemas encontrados y las soluciones aplicadas. Permiten hacer seguimiento del progreso del proyecto y sirven como evidencia del trabajo del equipo.

- 7. Organización del documento en normas APA 7:** Al documento se le aplicará el formato APA séptima edición, incluyendo la portada, las citas, las referencias y estructura en general. Esto nos ayuda a presentar un formato correcto académico del proyecto.
- 8. Diseño y contenido de las diapositivas:** Este entregable incluye la preparación de la presentación del proyecto para su sustentación, con diapositivas que resuman los objetivos, procesos, avances, resultados y conclusiones del sistema. Se cuidará el diseño visual, la claridad de la información y la coherencia con el resto del proyecto.
- 9. Resultado final del sistema:** Se hace entrega del sistema SaborUAM con todo su funcionamiento, buen diseño de su interfaz, base de datos configurada, documentación técnica completa y lista para que el cliente la utilice. En este caso para ser evaluada por la docente que está acompañando los proyectos.

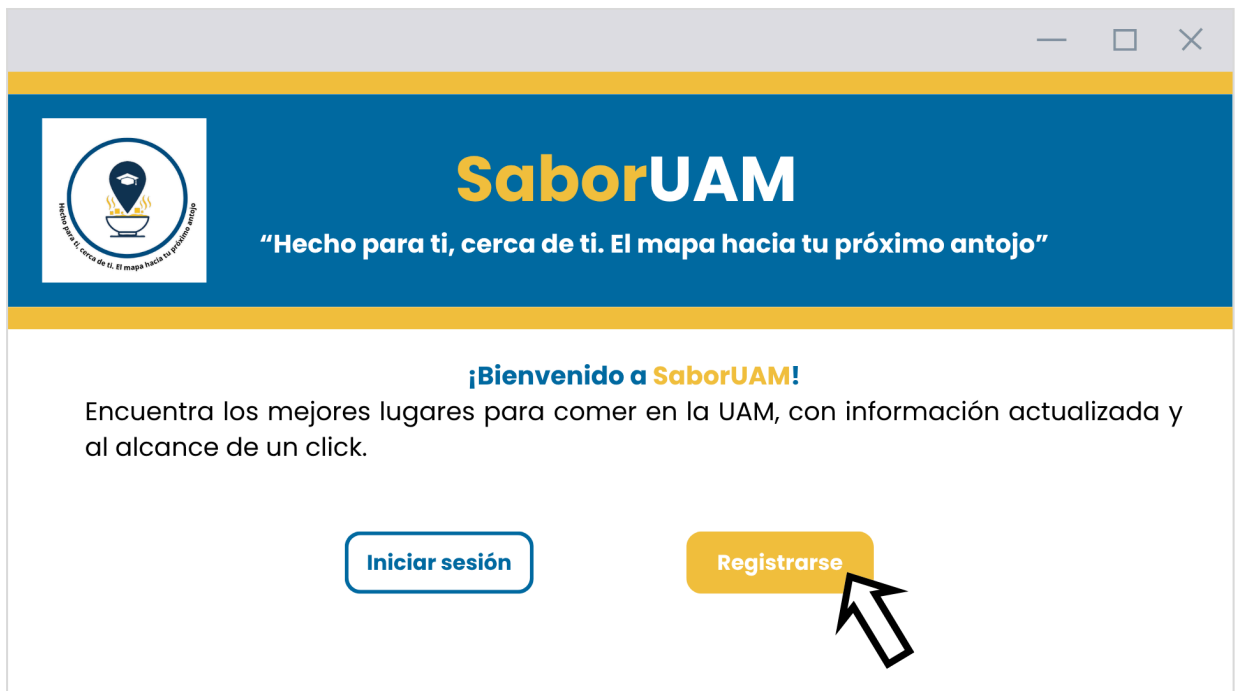
6.3. Cronograma

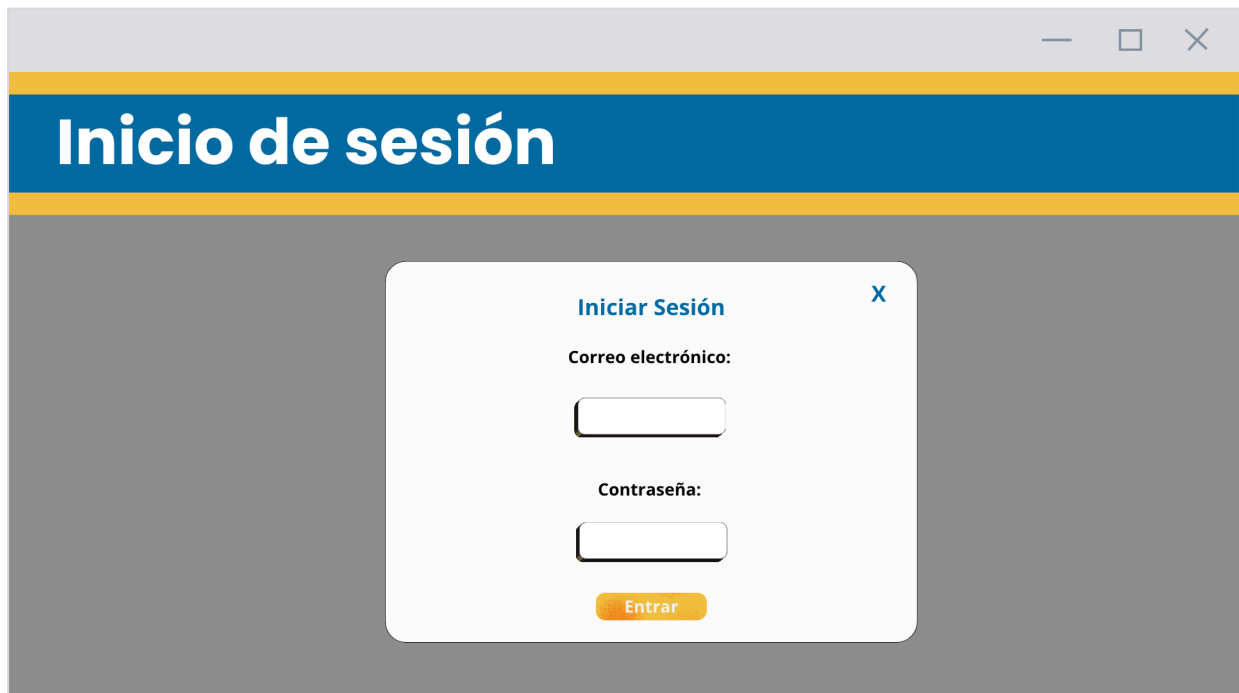
Cronograma del Proyecto SCRUM – “Sabor UAM”						
Sprint/ Semana	Fecha Inicio	Fecha Fin	Meta del Sprint	Actividades principales	Entregables	Estado
Semana 0 (Planeación inicial)	9/22/2025	10/1/2025	Definir el proyecto, asignar roles y seleccionar herramientas de trabajo colaborativo.	Elegir proyecto, asignar roles y organizar trabajo.	Proyecto elegido, roles definidos y herramientas establecidas	Completado
Sprint 1 – Análisis y planificación	10/2/2025	10/9/2025	Analizar los requerimientos y definir la base del sistema.	SCRUM-41: Objetivos del proyecto SCRUM-40: Justificación del proyecto SCRUM-43: Identificar Stakeholders SCRUM-45: Identificar casos de uso SCRUM-44: Diagrama casos de uso SCRUM-47: Lista de requerimientos funcionales SCRUM-48: Lista de requerimientos no funcionales SCRUM-52: Recolección de requisitos SCRUM-53: Especificación casos de uso	Documento y diagrama listos	Completado
Sprint 2 – Diseño visual y estructural	10/10/2025	10/17/2025	Diseñar la estructura y apariencia del sistema.	SCRUM-50: Cronograma SCRUM-54: Mockups e imágenes de las funcionalidades SCRUM-57: Logo SCRUM-58: Eslogan SCRUM-60: Descripción de entidades, atributos y tipos de datos Requerimientos funcionales: SCRUM-3, SCRUM-16, SCRUM-20, SCRUM-21, SCRUM-22, SCRUM-23, SCRUM-24, SCRUM-29, SCRUM-31, SCRUM-32 Requerimientos no funcionales: SCRUM-11, SCRUM-12, SCRUM-13, SCRUM-14, SCRUM-15, SCRUM-17, SCRUM-33, SCRUM-36, SCRUM-37, SCRUM-39	Historias de usuario, mockups, logo y diseño visual listos.	Completado

Sprint 3 – Diseño lógico y base de datos	10/18/2025	10/24/2025	Crear la base de datos y las interfaces del sistema.	SCRUM-69: Realizar bitácora del Sprint 3 SCRUM-62: Diseño de interfaz (agregar producto y vista de menú) SCRUM-61: Diseño de interfaz (registro e inicio de sesión) SCRUM-59: Relaciones entre entidades SCRUM-56: Diagrama entidad-relación SCRUM-49: Documento de visión SCRUM-26: Gestionar los productos del menú	Diagrama entidad-relación y prototipo de interfaz.	Completado
Sprint 4 – Desarrollo funcional	10/25/2025	10/31/2025	Desarrollar las funciones principales del sistema.	SCRUM-85: Crear base de datos SCRUM-72: Planificación de entregables SCRUM-71: Técnica de planificación elegida SCRUM-70: Realizar bitácora sprint 4 SCRUM-66: Conectar la base de datos con las interfaces SCRUM-65: Programar agregar productos al menú SCRUM-64: Programar el inicio de sesión y Ventana menú SCRUM-63: Programar el registro de usuarios	Sistema funcional conectado a la base de datos.	Completado
Sprint 5 – Pruebas y entrega final	11/1/2025	11/8/2025	Probar el sistema, corregir errores y preparar la entrega final.	SCRUM-87 – Slogan y logo (Diapositiva) SCRUM-86 – Realizar problema a solucionar (Diapositiva) SCRUM-82 – Realizar cómo funciona el sistema (Diapositiva) SCRUM-79 – Conclusiones (Diapositiva) SCRUM-76 – Mockups (Diapositiva) SCRUM-75 – Realizar objetivos (Diapositiva) SCRUM-73 – Realizar bitácora 5 SCRUM-68 – Realizar diseño diapositiva SCRUM-67 – Probar que cada formulario funcione correctamente SCRUM-42 – Organizar el proyecto normas APA 7	Proyecto probado, corregido y presentación lista.	Completado
Cierre del proyecto	11/9/2025	11/10/2025	Presentar el proyecto y evaluar el trabajo del equipo.	Presentación final.	Presentación final.	Completado

7. Evidencias del Desarrollo

7.1. Mockups e imágenes de las funcionalidades





[Cerrar sesión](#)





Almuerzo Ejecutivo 

\$12.000

Restaurante UAM

Estado: **Disponible**

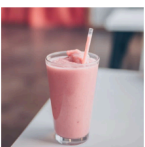


Almuerzo Vegetariano 

\$10.500

Verde Vida

Estado: **Disponible**



Jugo Natural 

\$4.000

Café UAM

Estado: **No disponible**

[Cerrar sesión](#)





Almuerzo Ejecutivo 

\$12.000

Restaurante UAM

Estado: **Disponible**

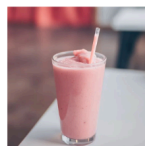


Almuerzo Vegetariano 

\$10.500

Verde Vida

Estado: **Disponible**



Jugo Natural 

\$4.000

Café UAM

Estado: **No disponible**

[Cerrar sesión](#)



☐ Desayunos

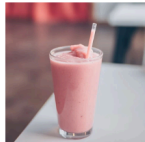
☐ Almuerzos

☐ Bebidas

☐ Otros productos



Almuerzo Vegetariano
\$10.500
Verde Vida
Estado: **Disponible**



Jugo Natural
\$4.000
Café UAM
Estado: **No disponible**

[Cerrar sesión](#)

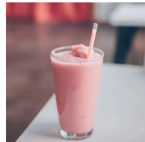




Almuerzo Ejecutivo
\$12.000
Restaurante UAM
Estado: **Disponible**

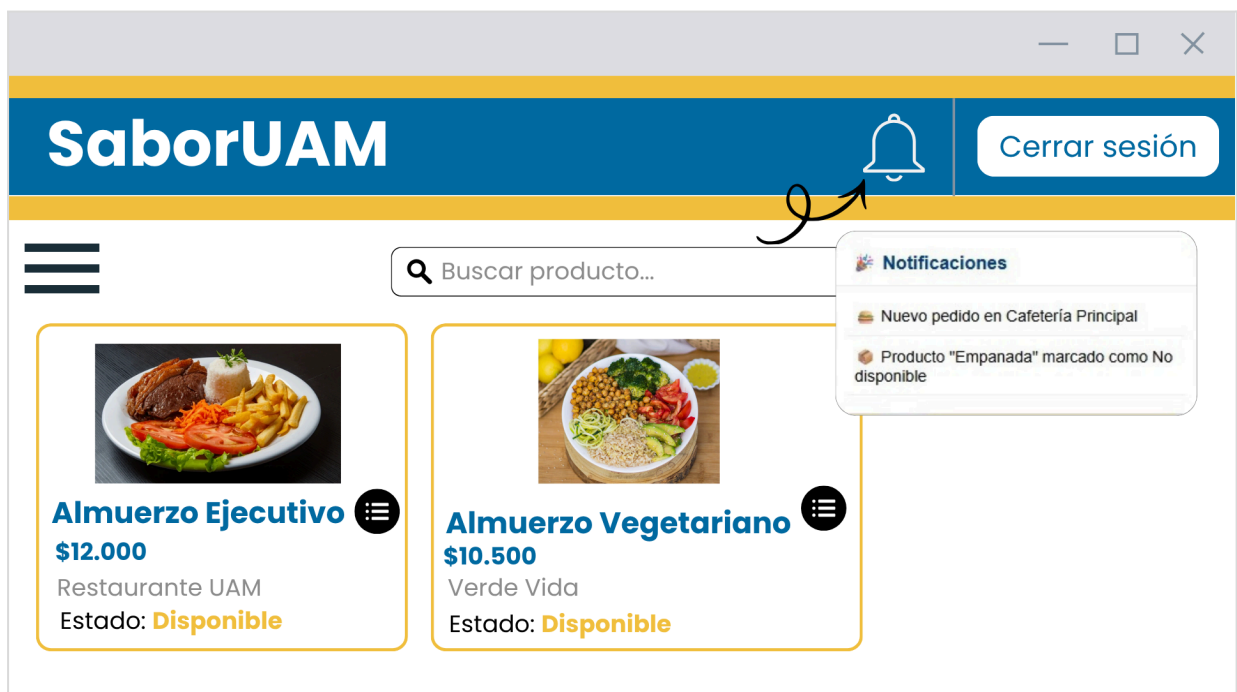
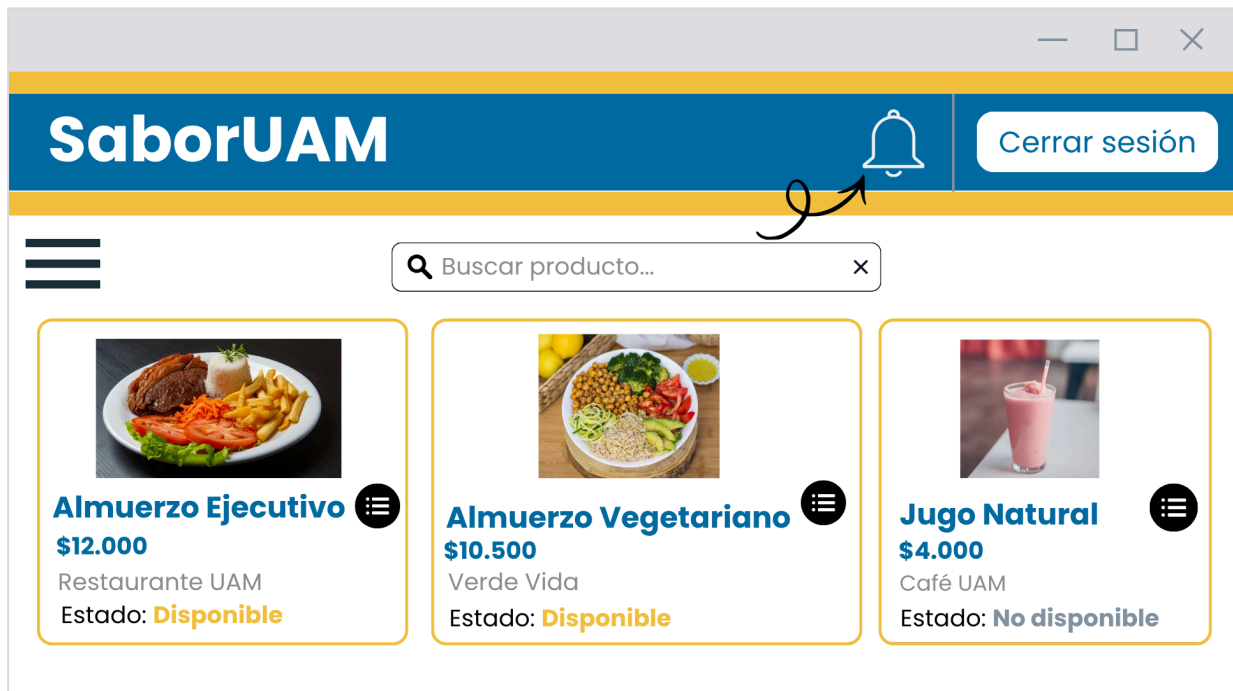


Almuerzo Vegetariano
\$10.500
Verde Vida
Estado: **Disponible**



Jugo Natural
\$4.000
Café UAM
Estado: **No disponible**





— □ ×

Administrador



Cerrar sesión



Almuerzo Ejecutivo 

\$12.000

Restaurante UAM

Estado: **Disponible**

Stock: 10



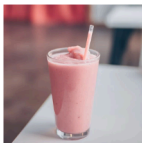
Almuerzo Vegetariano 

\$10.500

Verde Vida

Estado: **Disponible**

Stock: 10



Jugo Natural 

\$4.000

Café UAM

Estado: **No disponible**

Stock: 10



— □ ×

Administrador



Cerrar sesión



Almuerzo Ejecutivo 

\$12.000

Restaurante UAM

Estado: **Disponible**

Stock: 10

☐

Ver más

☐

Eliminar

☐

Actualizar





— □ ×

Administrador



Cerrar sesión



Almuerzo Ejecutivo 
\$12.000
Restaurante UAM
Estado: **Disponible**
Stock:

☐

Eliminar



— □ ×

Administrador



Cerrar sesión

**Confirmar eliminación
(simulación)**
¿Deseas eliminar el producto **Cocacola**?

Sí

No

— □ ×

Administrador

Cerrar sesión

🔍 Buscar producto... ×



Almuerzo Ejecutivo 

\$12.000

Restaurante UAM

Estado: **Disponible**

Stock: 10

☐

Ver más

☐

Eliminar

☐

Actualizar



— □ ×

Administrador

Cerrar sesión

🔍 Buscar producto... ×



Almuerzo Ejecutivo 

\$12.000

Restaurante UAM

Estado: **Disponible**

Stock: 10

☐

Actualizar



— □ ×

Administrador



Cerrar sesión

Editar producto

 Editar producto

Nombre

Precio

Tipo

— □ ×

Administrador



Cerrar sesión

Información del lugar



NIT

Nombre

Horario

Ubicación

Actualizar

Cancelar

— □ ×

Administrador



Cerrar sesión



Almuerzo Ejecutivo 

\$12.000

Restaurante UAM

Estado: **Disponible**

Stock: 10



Almuerzo Vegetariano 

\$10.500

Verde Vida

Estado: **Disponible**

Stock: 10

Notificaciones

 Nuevo pedido en Cafetería Principal

 Producto "Empanada" marcado como No disponible

— □ ×

Administrador



Cerrar sesión



Almuerzo Ejecutivo 

\$12.000

Restaurante UAM

Estado: **Disponible**

Stock: 10



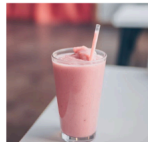
Almuerzo Vegetariano 

\$10.500

Verde Vida

Estado: **Disponible**

Stock: 10



Jugo Natural 

\$4.000

Café UAM

Estado: **No disponible**

Stock: 10



— □ ×

Administrador



Cerrar sesión

Agregar nuevo producto

Nombre del lugar:

Estado del lugar:

Selecciona una opción ▼

Tipo de lugar:

Selecciona una opción ▼

NIT:

Ubicación:

▲
▼

— □ ×

Administrador



Cerrar sesión

Agregar nuevo producto

Horario:

Días de atención:

¿Ofrece servicio a domicilio?

Selecciona una opción ▼

Número de contacto para domicilio:

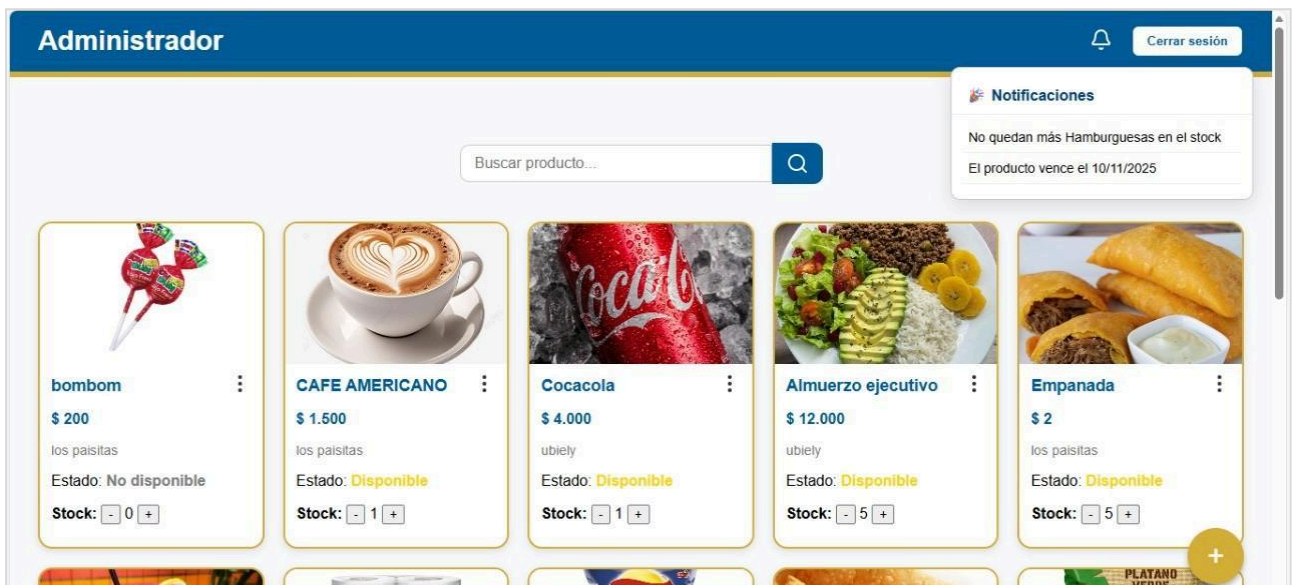
▲
▼

Atrás

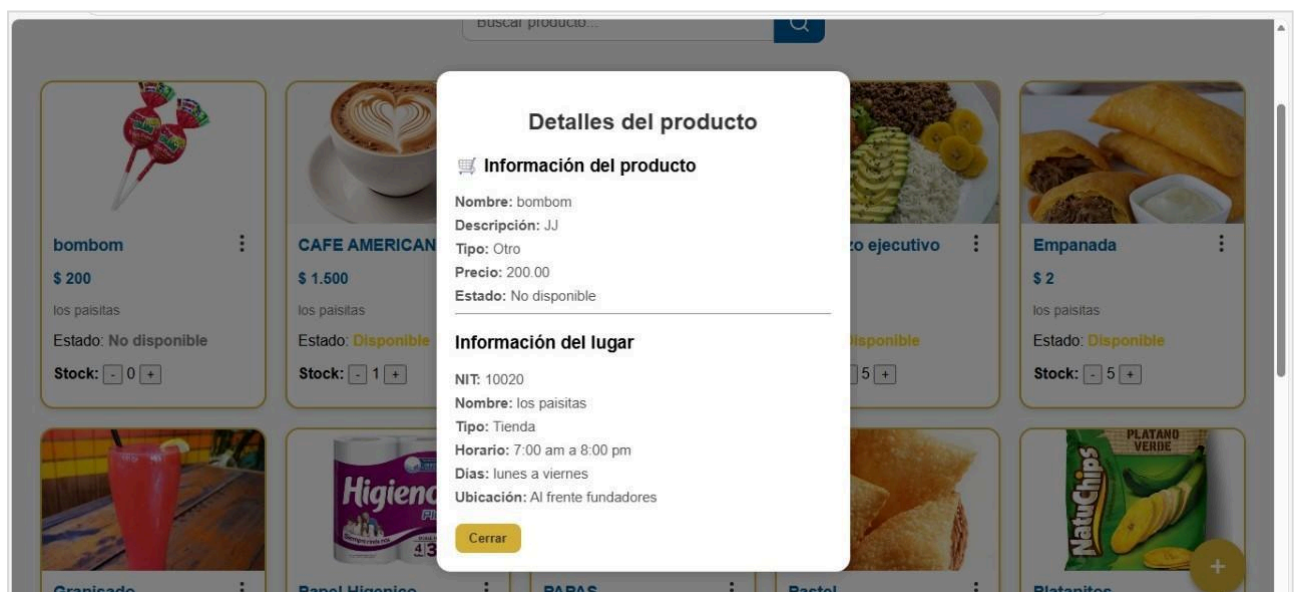
Agregar

7.2. Maquetación HTML de las funcionalidades

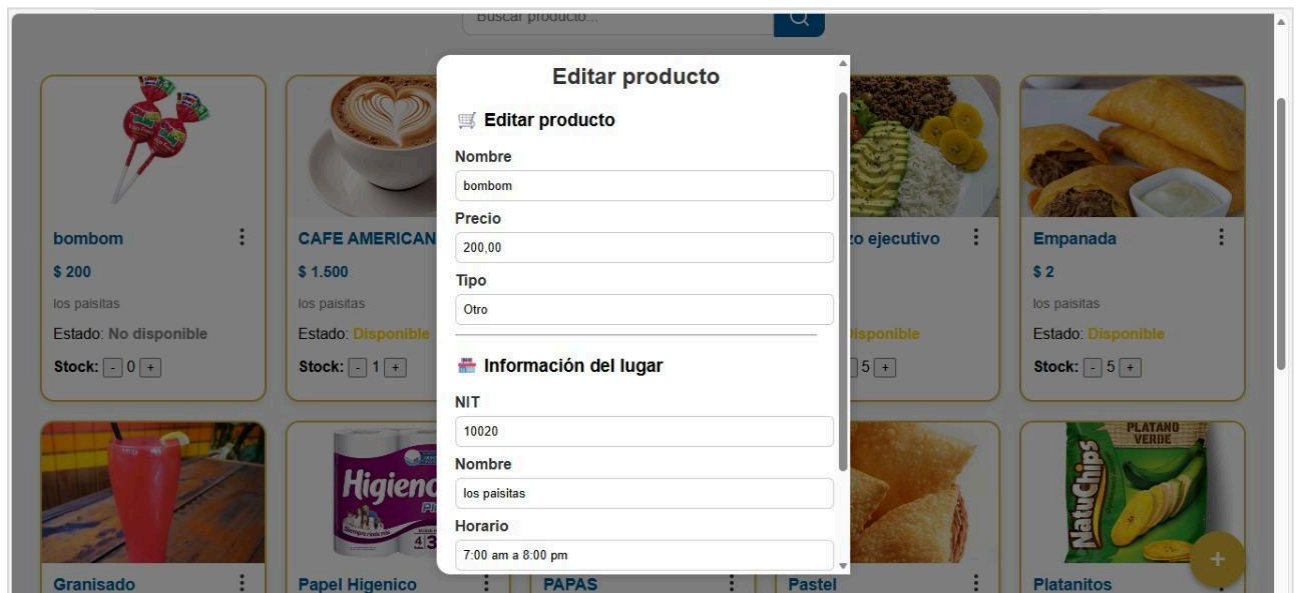
Notificaciones (Administrador)



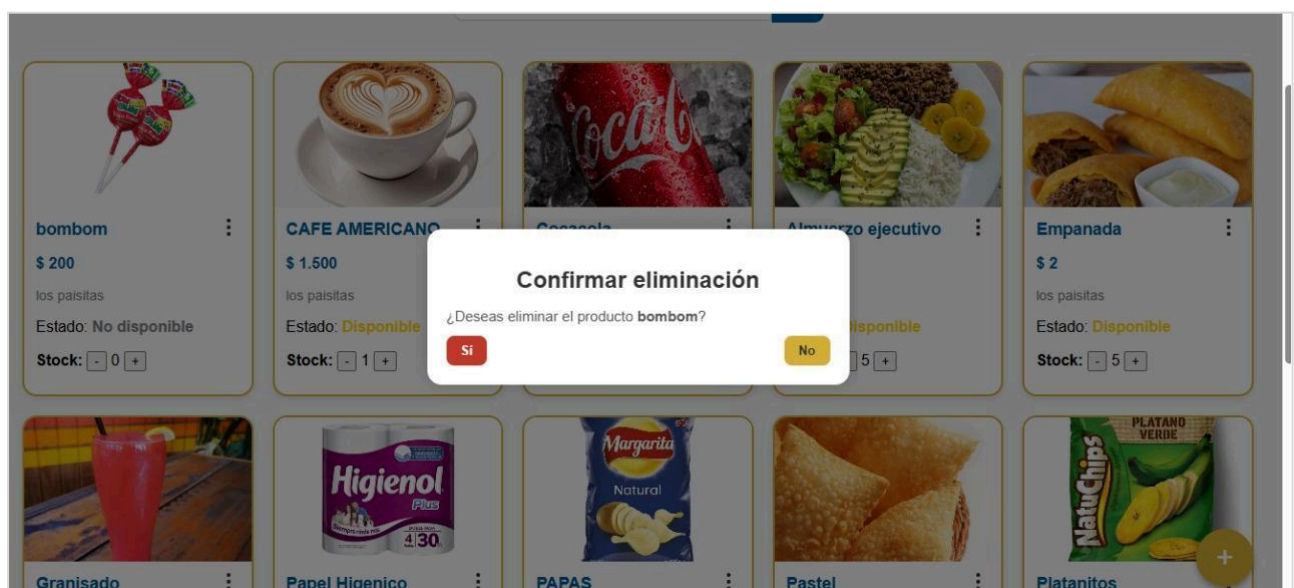
Ver más (Administrador)



Actualizar producto (Administrador)



Eliminar producto (Administrador)



Buscar producto (Administrador)

Administrador

 Cerrar sesión



Cocacola

\$ 4.000

ubieily

Estado: **Disponible**

Stock: 1



Página inicio (usuario)



SaborUAM

"Hecho para ti, cerca de ti. El mapa hacia tu próximo antojo"

¡Bienvenido a SaborUAM!

Encuentra los mejores lugares para comer en la UAM, con información actualizada y al alcance de un click.

Iniciar sesión

Registrarse

Iniciar sesión (usuario)



SaborUAM
"Hecho para ti, cerca de ti. El mapa hacia tu próximo antojo"

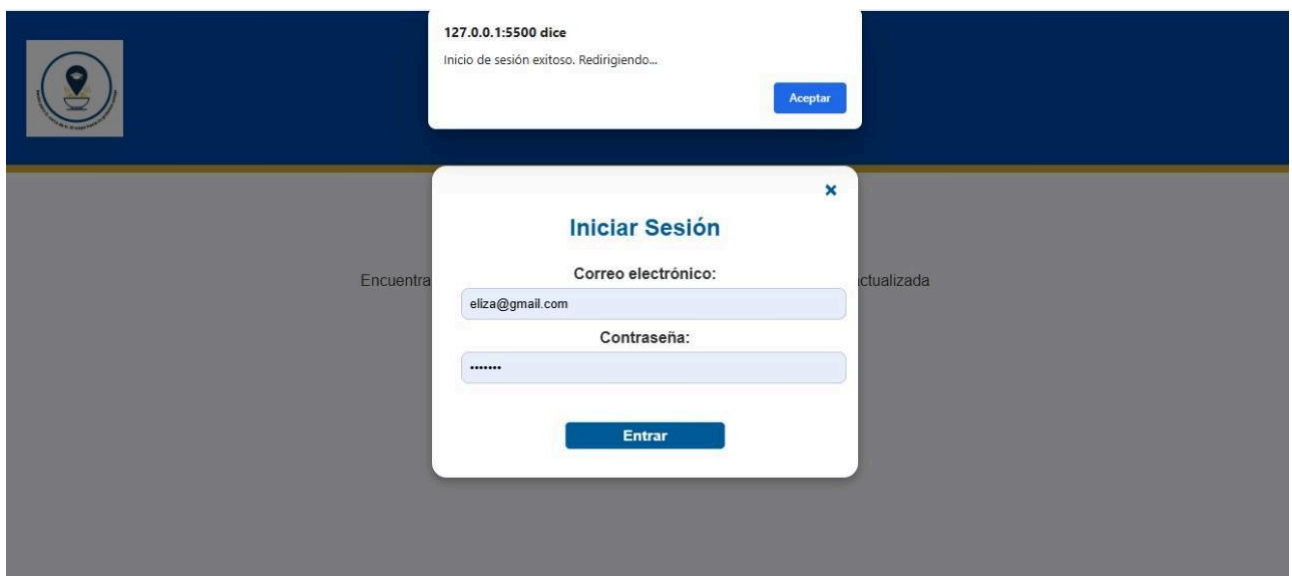
Iniciar Sesión

Correo electrónico:
eliza@gmail.com

Contraseña:

Entrar

Inicio exitoso (usuario)



127.0.0.1:5500 dice
Inicio de sesión exitoso. Redirigiendo...

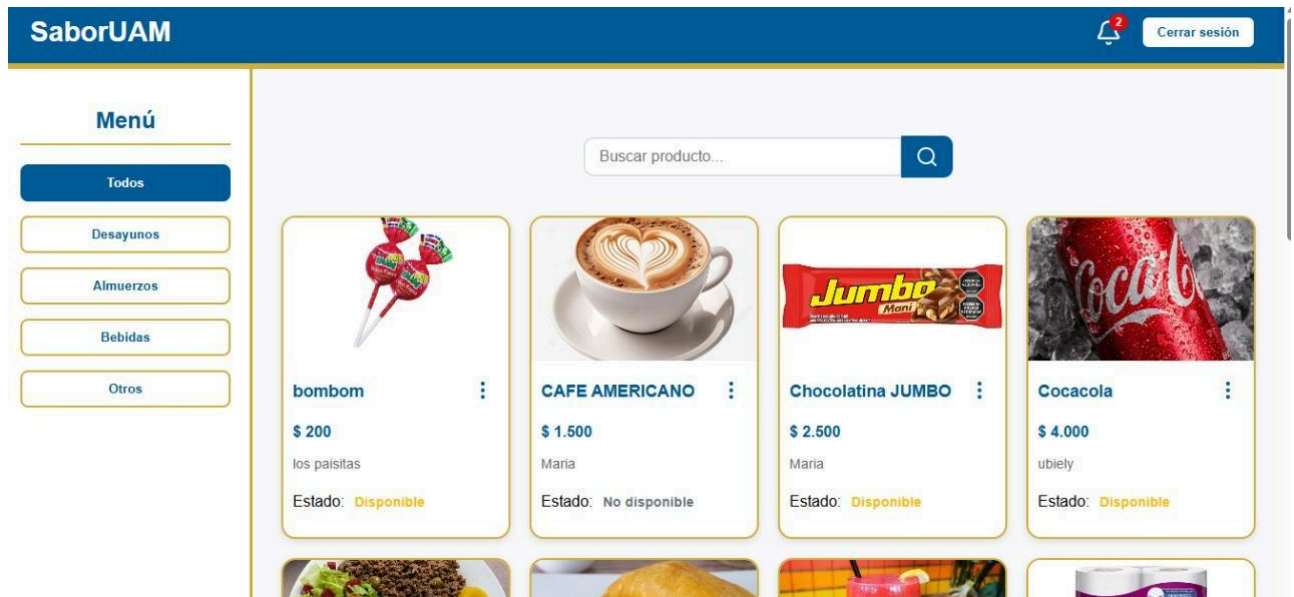
Aceptar

Iniciar Sesión

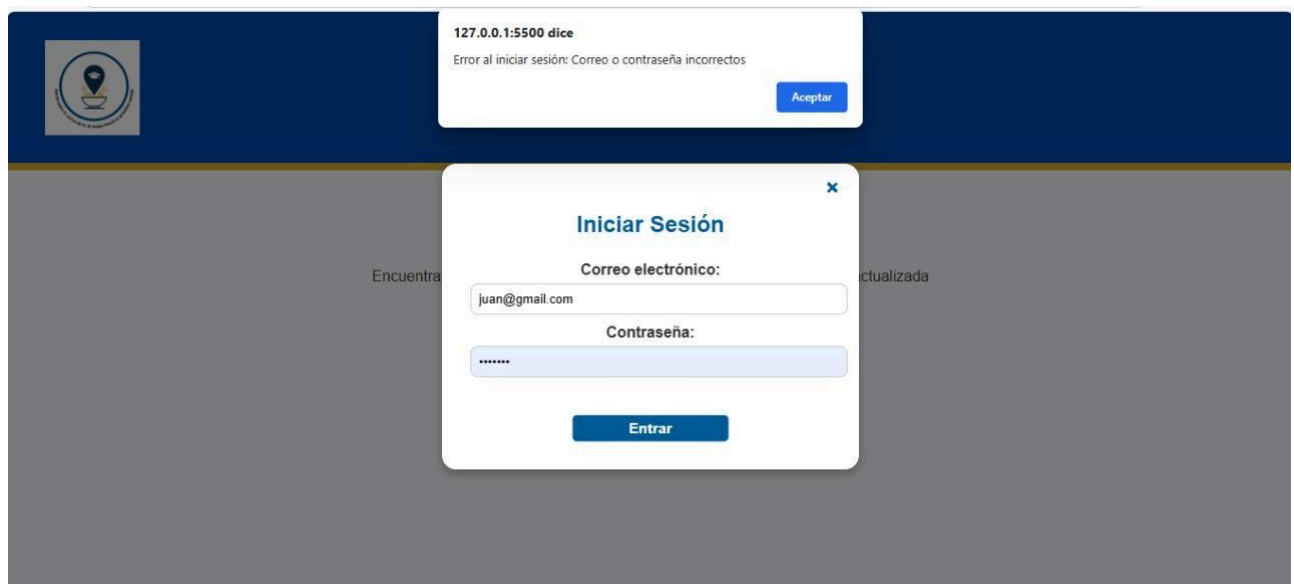
Correo electrónico:
eliza@gmail.com

Contraseña:

Entrar



Error al iniciar sesión (usuario)



Registrar (Usuario)

SaborUAM
"Hecho para ti, cerca de ti. El mapa hacia tu próximo antojo"

Encuentra actualizada

Registro de usuario ✕

Nombre completo:
elizabeth meneses

Tipo de documento:
Cédula de ciudadanía ▼

Número de documento:
12343545

Correo electrónico:
elizaeliza@autonoma.eu.co

Contraseña:

Registrar

Registro exitoso (usuario)

 127.0.0.1:5500 dice
Usuario registrado correctamente: Usuario registrado exitosamente

Encuentra actualizada

Registro de usuario

Nombre completo:
Maribel Ceballos

Tipo de documento:
Cédula de ciudadanía ▼

Número de documento:
1023456

Correo electrónico:
Maribel@gmail.com

Contraseña:

Registrar

Aceptar

7.3. Captura de código frontend relevante

Código (Inicio de sesión)

```
<!-- === MODAL DE LOGIN === -->
<!-- Ventana emergente que se muestra al hacer clic en "Iniciar sesión" -->
<section class="modal" id="modal-login" tabindex="-1" aria-hidden="true">
  <div class="modal-contenido">
    <!-- Botón (X) para cerrar la ventana -->
    <button type="button" class="btn-cerrar" id="cerrar-login" aria-label="Cerrar inicio de sesión">&times;</button>

    <!-- Título del modal -->
    <h2>Iniciar Sesión</h2>

    <!-- Formulario para ingresar credenciales del usuario -->
    <form id="form-login">
      <label for="email-login">Correo electrónico:</label>
      <input type="email" id="email-login" placeholder="Ej: heily@uam.edu.co" required>

      <label for="password-login">Contraseña:</label>
      <input type="password" id="password-login" placeholder="Mínimo 6 caracteres" required>

      <!-- Botón de envío -->
      <div class="acciones">
        <button type="submit" class="btn btn-entrar">Entrar</button>
      </div>
    </form>
  </div>
</section>
```

Código (Registro usuario)

```
// === REGISTRO ===

// Se obtiene el formulario de registro por su ID
const formRegistro = document.getElementById('form-registro');

// Verifica si el formulario existe en la página (evita errores si el script se carga en otra vista)
if (formRegistro) {
  // Escucha el evento "submit" (cuando el usuario intenta enviar el formulario)
  formRegistro.addEventListener('submit', async (e) => {
    e.preventDefault(); // Evita que la página se recargue al enviar el formulario

    // --- Captura de los valores ingresados en los campos del formulario ---
    const nombre_completo = document.getElementById('nombre_completo').value.trim();
    const tipo_de_documento = document.getElementById('tipo_de_documento').value;
    const numero_de_documento = document.getElementById('numero_de_documento').value.trim();
    const correo_electronico = document.getElementById('correo_electronico').value.trim();
    const contrasena = document.getElementById('contrasena').value;

    // --- Validación básica ---
    // Si algún campo está vacío, se muestra una alerta y se cancela el envío
    if (!nombre_completo || !tipo_de_documento || !numero_de_documento || !correo_electronico || !contrasena) {
      alert('Por favor completa todos los campos.');
```

```
      return;
    }

    try {
      // --- Envío de los datos al backend ---
      // Se realiza una petición HTTP POST al endpoint del servidor (API)
      const res = await fetch('http://localhost:3000/api/usuarios/registro', {
        method: 'POST', // Método HTTP
        headers: { 'Content-Type': 'application/json' }, // Indicamos que el cuerpo es JSON
        body: JSON.stringify({
```


Código usuarios (Envío de datos al backend)

```
try {
  // --- Envío de datos al backend ---
  // Se realiza una solicitud HTTP al servidor (API) para registrar un nuevo usuario
  const res = await fetch('http://localhost:3000/api/usuarios/registro', {
    method: 'POST', // Método HTTP: POST indica que se enviarán datos al servidor
    headers: { 'Content-Type': 'application/json' }, // Se especifica que el contenido es JSON
    body: JSON.stringify({
      // Se convierte el objeto con los datos del formulario a formato JSON
      tipo_de_documento,
      numero_de_documento,
      nombre_completo,
      correo_electronico,
      contrasena,
    }),
  });

  // --- Espera y procesa la respuesta del backend ---
  const data = await res.json(); // Convierte la respuesta JSON del servidor en un objeto JavaScript

  // --- Verifica si la respuesta fue exitosa ---
  if (res.ok) {
    // Si el servidor respondió correctamente (códigos 200-299)
    alert('Usuario registrado correctamente ');
    console.log('Respuesta del backend:', data); // Muestra la respuesta en consola (útil para depuración)
    formRegistro.reset(); // Limpia los campos del formulario después del registro
  } else {
    // Si el servidor devolvió un error (por ejemplo, error 400 o 500)
    alert('Error de registro: ' + (data.mensaje || 'No se pudo registrar el usuario'));
  }
} catch (error) {
  // --- Manejo de errores de conexión o ejecución ---
  // Este bloque se ejecuta si el servidor no responde o hay un fallo en la red
  console.error('Error al registrar usuario:', error);
}
```

Código (Agregar producto)

```
document.addEventListener('DOMContentLoaded', () => {

  // Modal agregar producto
  // Cuando se hace click en el botón "+" se abre el modal para agregar el nuevo producto.
  btnAgregar?.addEventListener('click', () => {
    modal.style.display = 'flex'; // Sirve para mostrar el modal.
    paso1.style.display = 'block'; // Sirve para mostrar el primer paso (Datos del producto).
    paso2.style.display = 'none'; // Sirve para ocultar el segundo paso (Datos del lugar).
  });

  // Cuando se hace click en el botón "Cancelar" se cierra el modal y limpia los campos.
  btnCancelar?.addEventListener('click', () => {
    modal.style.display = 'none';
    formProducto.reset();
  });

  // Al presionar el botón "Sigüiente", se validan los datos del primer paso.
  btnSigüiente?.addEventListener('click', () => {
    const nombre = document.getElementById('nombreProd').value.trim();
    const codigo = document.getElementById('codigo').value.trim();
    const tipo_menu = document.getElementById('tipo_menu').value.trim();
    const descripcion = document.getElementById('descripcion').value.trim();
    const precioStr = document.getElementById('precio').value.trim();
    const stockStr = document.getElementById('stock').value.trim();
    const archivo = inputImagen?.files?.[0];

    // Convierte los valores a números.
    const precio = precioStr !== '' ? Number(precioStr) : null;
    const stock = stockStr !== '' ? Number(stockStr) : null;

    // Si falta algún campo por llenar, se muestra el mensaje.
    if (!nombre || !codigo || !tipo_menu || !descripcion || precio === null || stock === null || !arch
```

```

    siguiente.addEventListener('click', () => {
        // Si falta algún campo por llenar, se muestra el mensaje.
        if (!nombre || !codigo || !tipo_menu || !descripcion || precio === null || stock === null || !ar
            alert('⚠ Por favor completa todos los campos antes de continuar.');
```

return;

}

// Si todo sale bien, se muestra el segundo paso (Datos del lugar).

paso1.style.display = 'none';

paso2.style.display = 'block';

});

// Si se apreta el botón "Atrás" se devuelve al primero paso (Datos del producto).

btnAtras?.addEventListener('click', () => {

paso2.style.display = 'none';

paso1.style.display = 'block';

});

// Guardar producto y lugar

// Cuando el formulario se envía, se guarda la información en la base de datos.

formProducto?.addEventListener('submit', async (event) => {

event.preventDefault();

// Se obtienen todos los valores de los campos del formulario.

const nombreProducto = document.getElementById('nombreProducto')?.value.trim();

const codigo = document.getElementById('codigo')?.value.trim();

const descripcion = document.getElementById('descripcion')?.value.trim();

const tipo_menu = document.getElementById('tipo_menu')?.value.trim();

const precio = document.getElementById('precio')?.value.trim();

const estadoProducto = document.getElementById('estadoProducto')?.value.trim();

```

formProducto?.addEventListener('submit', async (event) => {
    const precio = document.getElementById('precio')?.value.trim();
    const estadoProducto = document.getElementById('estadoProducto')?.value.trim();

    const nombreLugar = document.getElementById('nombreLugar')?.value.trim();
    const nit = document.getElementById('nit')?.value.trim();
    const ubicacion = document.getElementById('ubicacion')?.value.trim();
    const horario = document.getElementById('horario')?.value.trim();
    const dias = document.getElementById('dias')?.value.trim(); // <-- capturar días
    const servicioDomicilio = document.getElementById('servicioDomicilio')?.value.trim();
    const numeroContacto = document.getElementById('numeroContacto')?.value.trim();
    const estadoLugar = document.getElementById('estado')?.value.trim();
    const tipo = document.getElementById('tipo')?.value.trim();

    try {
        // Se guardar la información del lugar.
        const lugarData = {
            NIT: nit,
            nombre: nombreLugar,
            tipo,
            horario,
            estado: estadoLugar,
            servicioDomicilio,
            numeroContacto,
            ubicacion,
            dias
        };

        // Se le hace una petición al backend para agregar el nuevo lugar y "fetch" se utiliza para envia
        const resLugar = await fetch(`${API_URL}/api/lugares/registro`, {
            method: 'POST'
        });
    } catch (error) {
        console.log(error);
    }
});

```

```

    rmProducto?.addEventListener('submit', async (event) => {
        formData.append("imagen", inputImagen.files[0]);
        formData.append("NIT", nit);
        formData.append('stock', stock);

        // Se envia el producto al servidor usando fetch con método POST, luego el FormData se pasa en el
        const resProducto = await fetch(`${API_URL}/api/productos`, { method: 'POST', body: formData });

        // Se convierte la respuesta del servidor a formato JSON para poder leerla.
        const dataProducto = await resProducto.json();

        // Se verifica si la respuesta no fue exitosa y se lanza un error con el mensaje recibido.
        if (!resProducto.ok) throw new Error(dataProducto.error || 'Error al guardar el producto');

        // Si todo sale bien, se actualiza la vista
        alert('✅ Producto y lugar agregados correctamente.');
```

modal.style.display = 'none';
formProducto.reset();
paso1.style.display = 'block';
paso2.style.display = 'none';
cargarProductos();

```

        // Si algo sale mal, me lanza el error al guardar el producto o lugar.
    } catch (error) {
        console.error('❌ Error al guardar producto o lugar:', error);
        alert('Hubo un error al guardar. Revisa la consola.');
```

}

```

    ;

```

Admin.html

```

<!-- Modal agregar producto -->
<!-- Este cuadro aparece cuando el administrador hace click en el botón "+" -->
<section class="modal" id="modal-agregar" style="display: none;">
  <div class="modal-contenido">
    <h2>Agregar nuevo producto</h2>

    <!-- Es el formulario para agregar un nuevo producto -->
    <form id="form-producto">

      <!-- Paso 1: Se piden los datos básicos del nuevo producto -->
      <div class="paso" id="paso1">

        <!-- Son los espacios para pedir los datos básicos del nuevo producto a agregar -->
        <label for="nombreProducto">Nombre del producto:</label>
        <input type="text" id="nombreProducto" placeholder="Ej: Arepa con queso">

        <label for="codigo">Código:</label>
        <input type="text" id="codigo" placeholder="Ej: 64A90">

        <label for="descripcion">Descripción:</label>
        <textarea id="descripcion" placeholder="Descripción del producto..."></textarea>

        <!-- Se selecciona el tipo de menú al que pertenece -->
        <label for="tipo_menu">Tipo de menú:</label>
        <select id="tipo_menu">
          <option value="">Selecciona una opción</option>
          <option value="Desayuno">Desayuno</option>
          <option value="Almuerzo">Almuerzo</option>
          <option value="Bebida">Bebida</option>
          <option value="Otro">Otro</option>

```

```

<!-- Es el campo para poner el estado del lugar (si está activo o no) -->
<div class="campo">
  <label for="estado">Estado del lugar:</label>
  <select id="estado">
    <option value="">Seleccione un estado</option>
    <option value="Activo">Activo</option>
    <option value="Inactivo">Inactivo</option>
  </select>
</div>

<!-- Es el campo para elegir el tipo de lugar (Restaurante, Cafetería, etc.) -->
<div class="campo">
  <label for="tipo">Tipo de lugar:</label>
  <select id="tipo">
    <option value="">Seleccione un tipo</option>
    <option value="Restaurante">Restaurante</option>
    <option value="Cafetería">Cafetería</option>
    <option value="Tienda">Tienda</option>
    <option value="Otro">Otro</option>
  </select>
</div>

<!-- NIT del establecimiento -->
<label for="nit">NIT:</label>
<input type="text" id="nit" placeholder="Ej: 123456789-0">

```

```

<!-- Es el campo para poner el precio del producto -->
<label for="precio">Precio:</label>
<input type="number" id="precio" placeholder="Ej: 8000">

<!-- Stock disponible-->
<label for="stock">Stock inicial:</label>
<input type="number" id="stock" placeholder="Ej: 10" min="0">

<!-- Es el campo para subir la imagen del producto -->
<label for="imagen-producto">Imagen del producto:</label>
<input type="file" id="imagen-producto" accept="image/*">

<!-- Botones para cancelar o continuar al siguiente paso -->
<div class="acciones">
  <button type="button" id="btn-cancelar">Cancelar</button>
  <button type="button" id="btn-siguiente">Siguiente</button>
</div>
</div>

<!-- Paso 2: Se piden los datos del lugar donde se puede encontrar el nuevo producto -->
<div class="paso" id="paso2" style="display:none;">

  <!-- Aquí se piden los datos del lugar donde se vende el producto -->
  <label for="nombreLugar">Nombre del lugar:</label>
  <input type="text" id="nombreLugar" placeholder="Ej: Restaurante UAM">

```

```

<!-- Lugar exacto dentro del campus o zona -->
<label for="ubicacion">Ubicación:</label>
<input type="text" id="ubicacion" placeholder="Cafetería principal">

<!-- Horario de atención -->
<label for="horario">Horario:</label>
<input type="text" id="horario" placeholder="7:00 a.m - 11:00 a.m">

<!-- Días de atención -->
<label for="dias">Días de atención:</label>
<input type="text" id="dias" placeholder="Lunes a viernes">

<!-- Si ofrece servicio a domicilio -->
<label for="servicioDomicilio">¿Ofrece servicio a domicilio?</label>
<select id="servicioDomicilio">
  <option value="">Selecciona una opción</option>
  <option value="Sí">Sí</option>
  <option value="No">No</option>
</select>

<!-- Número de teléfono para domicilios -->
<label for="numeroContacto">Número de contacto para domicilio:</label>
<input type="text" id="numeroContacto" placeholder="Ej: 3101234567">

<!-- Botones para volver al paso anterior o enviar el formulario -->
<div class="acciones">
  <button type="button" id="btn-atras">Atrás</button>

```

```

    <button type="submit" id="btn-agregar-final">Agregar</button>
  </div>
</div>
</form>
</div>
</section>

<!-- === Scripts === -->
<!-- Es el archivo JavaScript donde está toda la lógica (mostrar productos, abrir el modal, etc.) -->
<script src="js/admin.js"></script>

<!-- Activa los íconos de Lucide o .css (sin esto no se ven los íconos) -->
<script>lucide.createIcons();</script>
</body>
</html>

```

7.4. Captura de código **backend** correspondiente

Código (Iniciar sesión)

```
17 // --- LOGIN DE USUARIO ---
18 router.post('/login', (req, res) => {
19   const { correo_electronico, contrasena } = req.body;
20
21   const sql = `
22     SELECT * FROM usuarios
23     WHERE correo_electronico = ? AND contrasena = ?
24   `;
25
26   conexion.query(sql, [correo_electronico, contrasena], (err, results) => {
27     if (err) {
28       console.error('Error al iniciar sesión:', err);
29       res.status(500).json({ mensaje: 'Error en el servidor' });
30     } else if (results.length === 0) {
31       res.status(401).json({ mensaje: 'Correo o contraseña incorrectos' });
32     } else {
33       res.json({
34         mensaje: 'Inicio de sesión exitoso',
35         numero_de_documento: results[0].numero_de_documento,
36         nombre_completo: results[0].nombre_completo
37       });
38     }
39   });
40 });
41
42 export default router;
```


Código (Registro de usuario)

```
// --- REGISTRO DE USUARIO ---
router.post('/registro', (req, res) => {
  // Extraemos los datos enviados desde el cliente (por ejemplo, desde un formulario)
  const { tipo_de_documento, numero_de_documento, correo_electronico, nombre_completo, contrasena } = req.body;

  // Verificamos que todos los campos estén presentes
  if (!tipo_de_documento || !numero_de_documento || !correo_electronico || !nombre_completo || !contrasena) {
    // Si falta alguno, respondemos con un código 400 (error del cliente)
    return res.status(400).json({ mensaje: 'Todos los campos son obligatorios' });
  }

  // Sentencia SQL para insertar un nuevo usuario en la tabla "usuarios"
  const sql = `
    INSERT INTO usuarios (tipo_de_documento, numero_de_documento, correo_electronico, nombre_completo, contrasena)
    VALUES (?, ?, ?, ?, ?)
  `;

  // Ejecutamos la consulta con los valores proporcionados por el cliente
  conexion.query(
    sql,
    [tipo_de_documento, numero_de_documento, correo_electronico, nombre_completo, contrasena],
    (err) => {
      // Si ocurre un error en la base de datos, lo mostramos en consola y devolvemos un error al cliente
      if (err) {
        console.error('Error al registrar usuario:', err);
        res.status(500).json({ mensaje: 'Error al registrar usuario' });
      } else {
        // Si todo va bien, respondemos con un mensaje de éxito
        res.json({ mensaje: 'Usuario registrado exitosamente' });
      }
    }
  )
}
```

Código producto (Actualizar stock)

```
// === RUTA PATCH: Actualizar stock de un producto específico (CORRECCIÓN) ===
// Esta ruta coincide con la solicitud PATCH /api/productos/stock del frontend.
router.patch("/stock", (req, res) => {
  // Nota: No se utiliza multer aquí ya que solo se actualizan datos JSON.
  const { codigo, nit, stock } = req.body;

  // 1. Validar datos
  if (!codigo || !nit || stock === undefined || stock === null) {
    return res.status(400).json({ error: "Faltan datos obligatorios (código, nit, o stock)" });
  }

  const stockInt = parseInt(stock) || 0;
  // Determinar el estado basado en el stock (si stock > 0, Disponible; si stock <= 0, No disponible)
  const nuevoEstado = stockInt > 0 ? 'Disponible' : 'No disponible';

  // 2. Consulta SQL para actualizar 'stock' y 'estado' en productos_lugares
  const sql = `
    UPDATE productos_lugares
    SET stock = ?, estado = ?
    WHERE pro_cod = ? AND lug_nit = ?
  `;

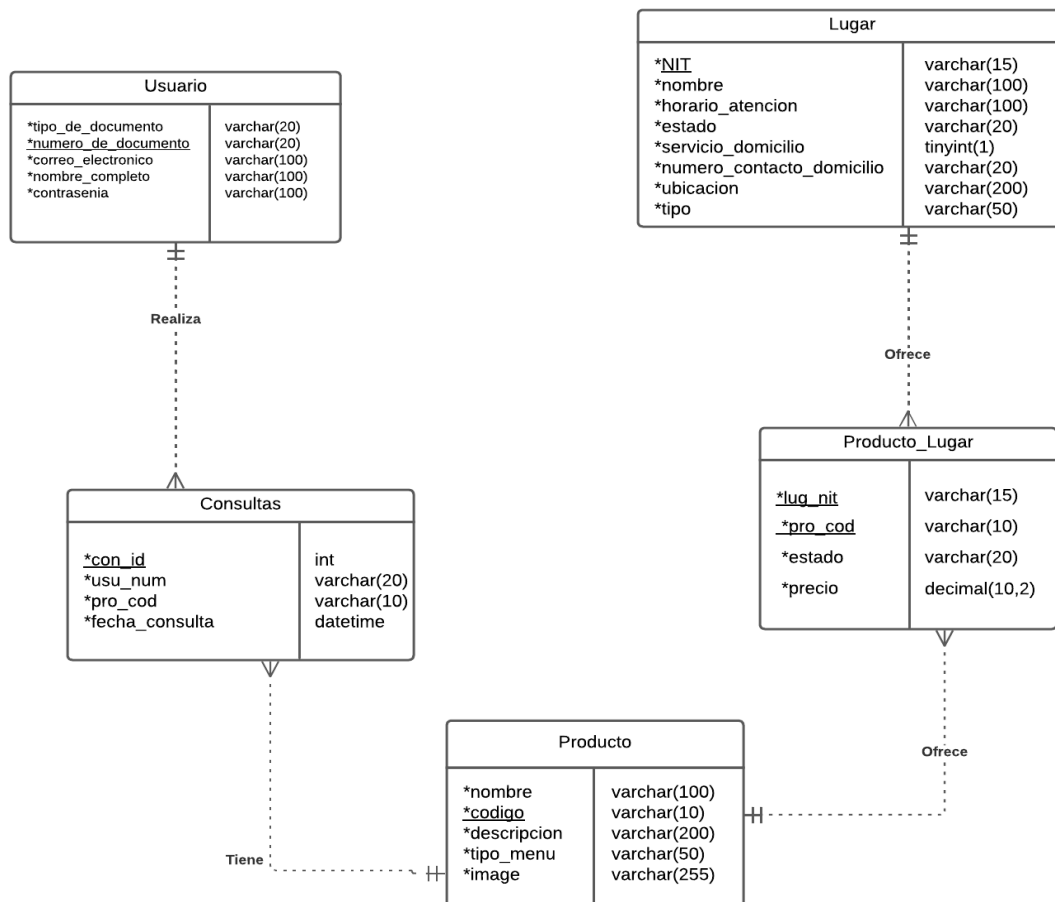
  db.query(sql, [stockInt, nuevoEstado, codigo, nit], (err, result) => {
    if (err) {
      console.error("❌ Error al actualizar el stock:", err);
      return res.status(500).json({ error: "Error en la base de datos al actualizar el stock" });
    }

    if (result.affectedRows === 0) {
      // Esto sucede si la relación pro_cod y lug_nit no existe
      return res.status(404).json({ error: "No se encontró la relación Producto-Lugar para actualizar." });
    }

    console.log(`✅ Stock de producto ${codigo} en lugar ${nit} actualizado a: ${stockInt}`);
    res.json({ mensaje: "Stock actualizado correctamente", nuevoStock: stockInt, nuevoEstado });
  });
});
```

8. Diseño de Base de Datos

8.1. Diagrama entidad-relación



8.2. Descripción de entidades, atributos y tipos de datos

En este punto se describen las entidades principales del sistema Sabor UAM, junto con sus atributos y tipos de datos. Estas entidades representan los elementos del sistema y la información que se almacena de cada uno.

Usuario:

Se identifica por su correo electrónico, ya que es único para cada persona. Guarda el nombre completo, el correo y la contraseña que usa para ingresar al sistema.

Lugar:

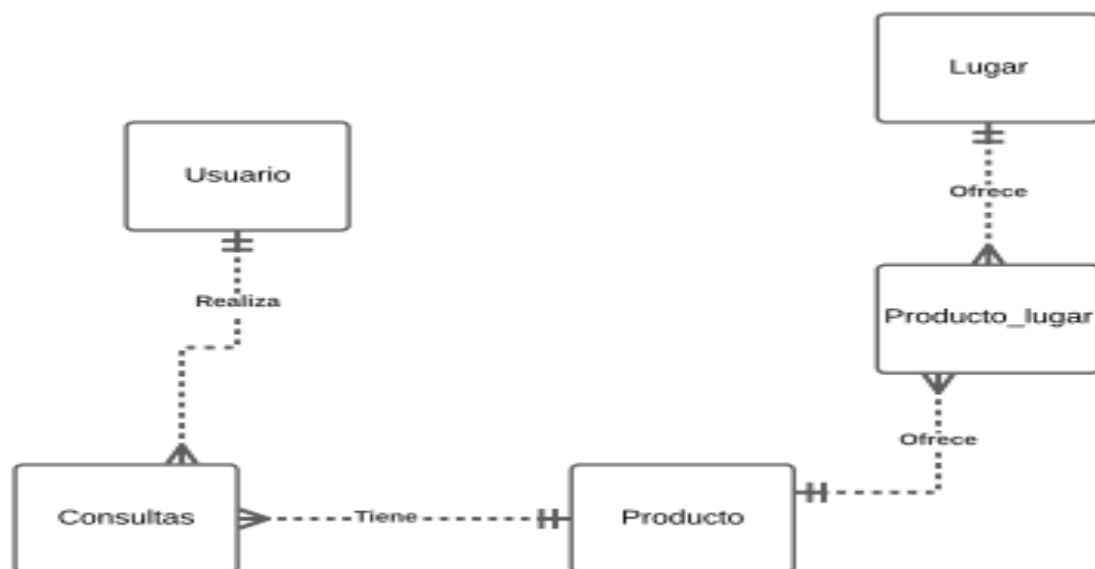
Se identifica por su nombre, ya que no se repite. También tiene ubicación, horario de atención y si ofrece servicio a domicilio; en ese caso, se muestra su número de contacto.

Producto:

Se identifica por el nombre del producto junto con el restaurante al que pertenece. Contiene la información del plato o bebida, como su descripción, precio, tipo de menú (desayuno, almuerzo u otros) y horario disponible.

8.3. Relaciones entre entidades

- Usuario y Consultas: Un usuario puede hacer múltiples consultas 1 a N
- Producto y Consultas: un producto puede tener varias consultas 1 a N
- Producto y Producto_Lugar: Un Producto puede ser ofrecido en múltiples
- Lugares (a través de la tabla intermedia) 1 a N
- Lugar y Producto_Lugar: Un Lugar ofrece múltiples Productos 1 a N



8.4. Script SQL con datos de prueba

9. Resultados y Presentación del Proyecto

9.1. Funcionalidades implementadas

9.2. Flujos de funcionamiento de la plataforma

9.3. Presentación en PowerPoint (evidencia de exposición en aula)

9.4. Infografía resumen del proyecto

10. Conclusiones

- El desarrollo del proyecto nos permitió aplicar todos los conocimientos vistos en clase como el método Scrum, que nos ayudó a organizar el trabajo de una manera eficiente y organizada por medio de los sprints, reuniones semanales y los diferentes roles que se asignaron (Scrum Master, Product Owner y Development team).
- La elaboración del proyecto permitió crear una página web funcional para conocer los lugares de alimentación disponibles, horarios, menús, entre otros. Donde gracias al trabajo de todos los miembros del equipo se logró dar una solución a la necesidad que se presenta en la UAM.

- El desarrollo de SaborUAM nos permitió hacer un proceso de investigación sobre el uso de HTML, CSS, JavaScript, MySQL, entre otras herramientas, para construir una página web funcional y agradable de ver visualmente para el usuario. Además, nos brindó la oportunidad de aprender cómo integrar el diseño, la lógica y la base de datos para realizar un sistema bien estructurado, dinámico y funcional.

Dificultades enfrentadas durante el desarrollo

- Trabajar en equipo fue un desafío, tuvimos que aprender a coordinar nuestras ideas y tareas.
- Integrar tecnologías como HTML, CSS, JavaScript y MySQL fue complicado, porque eran nuevas para nosotros.
- El diseño de la interfaz de usuario no salió como esperábamos, cometimos errores como duplicar código y no hacer que fuera consistente para diferentes roles.

10.2. Aprendizajes relevantes obtenidos

- Aprendimos a comunicarnos mejor y a trabajar en equipo de manera más efectiva.
- Aprendimos a investigar y a encontrar soluciones a problemas técnicos.
- Aprendimos a planificar y diseñar mejor la interfaz de usuario para que sea más fácil de usar y mantener.
- Aprendimos a ser más responsables con nuestras tareas y a separar nuestra vida personal de nuestras obligaciones académicas.

Observaciones adicionales

- La profesora fue de gran ayuda, nos guió y nos brindó apoyo durante todo el proyecto.
- Todos los desafíos y obstáculos que enfrentamos durante el proyecto nos serán de gran utilidad en nuestra vida profesional a futuro, ya que nos enseñaron a ser más resilientes, a trabajar en equipo y a encontrar soluciones creativas a problemas complejos.

11. Referencias Bibliográficas

- Mozilla Contributors. (2025, marzo 27). *Cómo funciona CSS*. MDN Web Docs. https://developer.mozilla.org/es/docs/Learn_web_development/Core/Styling_basics/What_is_CSS
- Mozilla Contributors. (2025, julio 21). *Conceptos básicos de HTML*. MDN Web Docs. https://developer.mozilla.org/es/docs/Learn_web_development/Getting_started/Your_first_website/Creating_the_content
- Alonso, S. (2023, diciembre 12). *Entendiendo la base del desarrollo web: HTML, CSS y JavaScript*. Dinahosting Blog. Recuperado el 31 de julio de 2025, de https://dinahosting.com/blog/entendiendo-la-base-del-desarrollo-web-html-css-y-javascript/#HTML_Estructura_y_contenido
- Casero, A. (2024, abril 8). *Crear una pantalla de login en HTML*. KeepCoding. <https://keepcoding.io/blog/crear-una-pantalla-de-login-en-html/>

- Casero, A. (2025, junio 5). *Crear un formulario de registro en HTML*. KeepCoding.
<https://keepcoding.io/blog/crear-un-formulario-de-registro-en-html/>